

1. Предложение WHERE. Базовый синтаксис.

При выборе информации из таблиц обычно требуются не все записи, а только их часть, удовлетворяющая какому-либо условию.

Оператор **SELECT** позволяет указать условия отбора записей для включения их в resultset. Делается это при помощи предложения **WHERE**.

В операторе **SELECT** предложение **WHERE** следует сразу за предложением **FROM**:

```
SELECT select_list  
FROM table(s)  
WHERE search_conditions
```

Здесь **search_conditions** - это условия отбора записей, которые могут включать в себя имена полей, константы, выражения, операторы сравнения и логические операторы.

При выполнении оператора **SELECT** с предложением **WHERE** в resultset включаются только записи, удовлетворяющие условиям **search_conditions**.

2. Операторы сравнения.

При записи условий отбора записей могут использоваться следующие операторы сравнения:

=	равно
>	левый операнд больше правого
<	левый операнд меньше правого
>=	левый операнд больше либо равен правому
<=	левый операнд меньше либо равен правому
!>	левый операнд не больше правого (аналог <=)
!<	левый операнд не меньше правого (аналог >=)
<>, !=	не равно
BETWEEN ... AND ...	попадание значения во включающий диапазон
[NOT] IN (список значений)	вхождение (или не вхождение) значения в список
[NOT] LIKE	совпадение (или не совпадение) с шаблоном
IS [NOT] NULL	сравнение с NULL

Режим сравнения символьных строк зависит от настроек сервера БД. Обычно при сравнении не учитывается регистр (т.е., 'ИВАНОВ' = 'иванов' = 'Иванов'), а также игнорируются пробелы в конце строки (т.е., 'Иванов' = 'Иванов ').

Примеры.

```
-- Выбрать всех клиентов с фамилией Иванов:
SELECT * FROM Clients WHERE LastName = 'Иванов'

-- Выбрать всех клиентов, родившихся в 1980-м году:
SELECT *
FROM Clients
WHERE DateBorn BETWEEN '1980-01-01' AND '1980-12-31'

-- Выбрать платежи в гривне (код UAH) или долларах США (код USD):
SELECT *
FROM Payments
WHERE Currency IN ('UAH', 'USD')

-- Выбрать всех клиентов с незаполненной датой рождения:
SELECT *
FROM Clients
WHERE DateBorn IS NULL
```

3. Оператор LIKE.

Оператор **LIKE** используется при необходимости провести поиск строковых значений по шаблону. В качестве источника шаблона могут выступать константы, переменные, выражения или поля таблиц.

Общий синтаксис оператора LIKE:

```
valueToCheck [NOT] LIKE template [ESCAPE escapeChar]
```

Здесь

- **valueToCheck** - поле таблицы, константа или выражение, значение которого проверяется на соответствие шаблону
- **template** - строка-шаблон
- **escapeChar** - символ, позволяющий маскировать служебные символы в шаблоне

Шаблон представляет из себя текстовую строку, в которой могут содержаться следующие служебные символы:

% (процент)	Любое количество любых символов
_ (подчеркивание)	Один любой символ
[спецификатор]	Попадение символа в диапазон значений. Спецификатор может указываться двумя способами: • диапазон символов с указанием начального и конечного символа диапазона, разделенных символом '-', например, [a-f] • перечень значений, например, [a2RI7]
[^спецификатор]	Символ '^', указанный перед спецификатором, указывает на "не вхождение". Например, [^a-f] означает "любые символы, кроме входящих в диапазон от a до f", [^2345] - "что угодно, кроме символов 2, 3, 4 и 5".

Примеры использования LIKE для выборки по фамилии:

- LastName LIKE 'ИВ%' найдет все фамилии, начинающиеся на 'ИВ': Иванов, Ивлев
- LastName LIKE '%цев' найдет все фамилии, оканчивающиеся на 'цев': Звягинцев, Кормильцев
- LastName LIKE '%чук%' найдет все фамилии, содержащие подстроку 'чук': Шевчук, Кравчуков, Чукалов
- LastName LIKE '_евченко' найдет все фамилии длиной 8 символов, последние 7 из которых - 'евченко': Шевченко, Левченко
- LastName LIKE '[MC]a[вр]чук' найдет фамилии Марчук и Савчук (например)
- LastName LIKE 'M[^ae]%' найдет все фамилии, начинающиеся на 'M', вторая буква в которых - не 'a' и не 'e'

При помощи **LIKE** можно также обрабатывать значения типа **datetime**. При этом сервер неявно преобразует дату и время в строковый формат. Например, поиск платежей, совершенных в **12:39** можно реализовать так:

```
SELECT * FROM Payments WHERE PayDT like '%12:39%'
```

Следует помнить, что по умолчанию сервер преобразует даты в строку формата **'Mon DD YYYY hh:mm'**. Кроме того, хотя такой способ и допустим, лучше им не пользоваться, поскольку он далеко не идеален с точки зрения производительности.

Если нужно произвести поиск строки, содержащей служебный символ, используется ключевое слово **ESCAPE** с указанием маскирующего (escape-) символа. Escape-символ не учитывается транслятором SQL, а следующий за ним символ строки интерпретируется как обычный, не служебный. Escape-символ может замаскировать и сам себя.

Например,

```
SomeField LIKE '%###_#%' escape '#'
```

ищет все строки, начинающиеся с символов **%#_#**. Первый, третий, пятый, седьмой символы "изымаются" из строки, маскируя второй, четвертый, шестой и восьмой, которые воспринимаются как неслужебные. Последний символ **'%'** не замаскирован, поэтому трактуется как служебный.

4. Сложные условия.

В предложении WHERE может быть записано сложное условие, включающее в себя несколько частей, объединенных логическими операторами

AND	Возвращает TRUE, если выполняются оба условия, объединяемых данным оператором
OR	Возвращает TRUE, если выполняется хотя бы одно из объединяемых условий
NOT	Возвращает TRUE, если условие, следующее за данным оператором, не выполняется

Приоритет выполнения операторов:

- умножение и деление
- сложение и вычитание
- операторы сравнения
- **NOT**
- **AND**
- **OR**

Если нужно указать последовательность выполнения явно, используются скобки.

Примеры.

```
-- Выбрать платежи во всех валютах,  
-- кроме гривны (UAH) и долларов США ( USD) :  
SELECT *  
FROM Payments  
WHERE Currency NOT IN ('UAH', 'USD')
```

```
-- Выбрать всех клиентов, фамилии которых начинаются на 'A',  
-- родившихся в 1980 году:  
SELECT *  
FROM Clients  
WHERE LastName LIKE 'A%'  
      AND DateBorn BETWEEN '1980-01-01' AND '1980-12-31'
```

```
-- Выбрать всех клиентов, родившихся в 1980 году,  
-- фамилии или имена которых начинаются на 'A'  
SELECT *  
FROM Clients  
WHERE ( FirstName LIKE 'A%' OR LastName LIKE 'A%' )  
      AND DateBorn BETWEEN '1980-01-01' AND '1980-12-31'
```

5. Объединение таблиц.

Зачастую данные нужно выбирать не из одной таблицы, а из нескольких, описывая взаимосвязи между этими таблицами.

Рассмотрим структуру, состоящую из двух таблиц - клиенты (**Clients**) и счета (**Accounts**). Таблица **Clients** содержит поле **ClientID** - идентификатор клиента. Таблица **Accounts** содержит аналогичное поле, позволяющее определить, какому клиенту принадлежит счет.

Clients

ClientID	FirstName	LastName	DateBorn
1	Иван	Иванов	1980-12-10
2	Петр	Петров	1976-05-08

Accounts

AccountID	ClientID	Currency	DateOpen
26007111111001	1	UAH	2011-10-12
26003111111002	1	USD	2011-11-01
26004222222001	2	UAH	2010-04-11

Требуется выбрать информацию обо всех счетах с отображением полей "Номер счета", "Код валюты", "Фамилия клиента", "Имя клиента":

AccountID	Currency	LastName	FirstName
26007111111001	UAH	Иванов	Иван
26003111111002	USD	Иванов	Иван
26004222222001	UAH	Петров	Петр

Чтобы сформировать подобную выборку, серверу нужно указать полный список таблиц, из которых нужно получить информацию, а также описать правила, по которым должны связываться записи из перечисленных таблиц.

Для описания объединений таблиц Sybase ASE поддерживает два варианта синтаксиса: **ANSI** и **Transact SQL**.

При использовании синтаксиса **ANSI** взаимосвязи между таблицами описываются в предложении **FROM**:

```
SELECT select_list
FROM table1 JOIN table2 ON join_conditions1
    [JOIN table3 ON join_conditions2 ...]
```

Для нашего примера оператор **SELECT** будет выглядеть так:

```
SELECT a.AccountID, a.Currency, c.LastName, c.FirstName
FROM Clients c JOIN Accounts a ON a.ClientID = c.ClientID
```

Второй способ описания объединения - использовать синтаксис **Transact SQL**. При этом условия объединения описываются в предложении **WHERE**:

```
SELECT select_list
FROM table1, table2 [, ... tableN]
WHERE (N-1) join_conditions
```

Для нашего примера запрос будет выглядеть так:

```
SELECT a.AccountID, a.Currency, c.LastName, c.FirstName
FROM Clients c, Accounts a
WHERE a.ClientID = c.ClientID
```

В варианте **ANSI** предложение **WHERE** так же может использоваться, но в этом случае обычно присутствует "разделение ролей": в предложении **FROM** описываются условия объединения таблиц, а в предложении **WHERE** - условия фильтрации данных. Для варианта **Transact SQL** и то, и другое описывается в предложении **WHERE**.

Например, если в нашу выборку нужно включить только счета, открытые в гривне, вариант **ANSI** будет выглядеть так:

```
SELECT a.AccountID, a.Currency, c.LastName, c.FirstName
FROM Clients c JOIN Accounts a ON a.ClientID = c.ClientID
WHERE a.Currency = 'UAH'
```

а вариант **Transact SQL** так:

```
SELECT a.AccountID, a.Currency, c.LastName, c.FirstName
FROM Clients c, Accounts a
WHERE a.ClientID = c.ClientID
AND a.Currency = 'UAH'
```

Теоретически, с точки зрения выполнения запроса, оба варианта записи аналогичны. Однако пути оптимизатора неисповедимы, поэтому зачастую путем перехода от одного варианта записи к другому удастся добиться более оптимального выполнения. Хотя обычно это касается более сложных запросов.

Принадлежность поля той или иной таблице определяется автоматически, если имя поля в используемых таблицах встречается только один раз. Если имя поля встречается в нескольких таблицах, участвующих в выборке (в нашем случае это ClientID), перед именем поля обязательно должно указываться имя соответствующей таблицы или ее псевдоним.

Хорошим тоном считается при выборке из нескольких таблиц явно указывать принадлежность всех полей, даже если их имена уникальны. Так запрос проще читать и понимать.

6. Декартово произведение.

Для понимания выполнения объединений таблиц на упрощенном уровне можно считать, что сервер выбирает записи из всех таблиц, перечисленных в предложении FROM, комбинирует их по принципу "каждая с каждой", но в окончательный resultset включает только те комбинации, которые удовлетворяют условиям объединения.

Естественно, оптимизатор работает более "умно", но общий принцип верен.

Исходя из описанного принципа становится понятно, что в условиях объединения должны быть описаны правила для КАЖДОЙ из используемых таблиц. Если в выборке участвует N таблиц, должно быть описано (N-1) условие объединения.

Если правило объединения таблиц не описать (или описать неправильно), в resultset будут включены все комбинации записей из используемых таблиц.

Так, для нашего примера, если не описать условия объединения, получим следующую выборку:

```
SELECT a.AccountID, a.Currency, c.LastName, c.FirstName  
FROM Clients c, Accounts a
```

AccountID	Currency	LastName	FirstName
26007111111001	UAH	Иванов	Иван
26007111111001	UAH	Петров	Петр
26003111111002	USD	Иванов	Иван
26003111111002	USD	Петров	Петр
26004222222001	UAH	Иванов	Иван
26004222222001	UAH	Петров	Петр

Как видно, количество записей в полученном resultset вычисляется как произведение количества записей в используемых таблицах. Такой эффект называется "**декартово произведение**". Легко представить, что будет, если не описать условие объединения для таблиц, содержащих по миллиону записей.

7. Примеры объединений.

В предыдущих разделах был рассмотрен пример объединения, основанный на равенстве значений полей в объединяемых таблицах. Существуют и другие варианты условий объединения таблиц, рассмотрим некоторые из них.

Попадание в диапазон значений.

Условия объединения могут быть более сложными и основываться, например, на попадании значения поля из одной таблицы в **диапазон значений**, описываемых полями другой таблицы.

Примером такого объединения может служить запрос, отображающий информацию об устройствах (device) Sybase ASE, на которых расположены базы данных.

В базе master существуют следующие таблицы:

- **sysdevices** - перечень устройств, используемых данным сервером
- **sysdatabases** - перечень баз данных
- **sysusages** - сведения о том, на каких устройствах расположена каждая база данных

Таблица **sysdevices** содержит поля **low** и **high**, содержащие номера начальной и конечной логических страниц, расположенных на данном устройстве.

Первичным ключом таблицы **sysdatabases** является поле **dbid** - идентификатор базы данных. По этому ключу можно получить расширенную информацию о базе, например, ее наименование (поле name).

Каждая запись таблицы **sysusages** содержит информацию о "кусочке" устройства, выделенном под конкретную базу данных. Эта таблица также содержит поле **dbid**, на основании которого можно понять, о какой базе идет речь, а также поле **size**, определяющее размер "кусочка" и поле **vstart**, содержащее начальный номер логической страницы, с которой он начинается. "Кусочек", описываемый записью из **sysusages** расположен на устройстве, описываемой записью из **sysdevices**, если поле **sysusages.vstart** попадает в диапазон от **sysdevices.low** до **sysdevices.high**.

Ниже приведены фрагменты данных из этих таблиц, для лучшего понимания картины.

sysdevices

name	low	high	...
master_dev	0	15359	...
device1_dat	67108864	83886079	...
device2_dat	83886080	100663295	...

sysdatabases

name	dbid	...
master	1	...
ips	7	...
oper	10	...

sysusages

dbid	size	vstart	...
1	3072	4	...
7	524288	68520460	...
10	131072	83886080	...
7	131072	90100512	...

Допустим, стоит задача сформировать выборку, отображающую информацию о том, сколько места с каждого устройства используется каждой базой данных. Причем в выборку должны попасть поля "Имя базы данных", "Имя устройства", "Размер части устройства, выделенного под данную базу".

db.name	d.name	u.size
master	master_dev	3072
ips	device1_dat	524288
oper	device2_dat	131072
ips	device2_dat	131072

Запрос, при помощи которого можно сформировать такую выборку, в варианте **ANSI** выглядит так:

```
SELECT db.name, d.name, u.size
FROM master..sysdatabases db
      JOIN master..sysusages u ON db.dbid = u.dbid
      JOIN master..sysdevices d ON u.vstart BETWEEN d.low AND d.high
```

С использованием синтаксиса **Transact SQL** этот же запрос выглядит так:

```
SELECT db.name, d.name, u.size
FROM master..sysdatabases db, master..sysusages u, master..sysdevices d
WHERE db.dbid = u.dbid
      AND u.vstart BETWEEN d.low AND d.high
```

Самообъединение.

Частным случаем объединения таблиц является **самообъединение**. При этом в соответствие

одной записи таблицы ставится другая запись из той же таблицы.

В качестве примера рассмотрим таблицу **sysprocesses** из базы **master**, содержащую информацию о текущих процессах на сервере.

spid	...	suid	dbid	blocked
1	...	1	7	2
2	...	1	7	0
3	...	2	15	0
...	...	...	...	...
48		8	7	2

Нас интересуют поля **spid** - ID процесса, первичный ключ таблицы **sysprocesses**, и **blocked** - ID процесса, блокирующего выполнение процесса, описываемого данной записью. В приведенном примере процесс с ID 2 блокирует процессы с ID 1 и 48.

Кроме перечисленных полей в **sysprocesses** присутствуют поля **status** (состояние), **hostname** (имя компьютера, с которого запущен процесс), **cmd** (команда, которую выполняет процесс). Эти поля мы используем в своем запросе.

Итак, для того, чтобы одним запросом вывести подробности как о заблокированном, так и о блокирующем процессе, нужно выполнить объединение записей из одной и той же таблицы:

```
SELECT "Процесс " + p.spid + " в состоянии " + p.status +  
      " блокируется процессом " + b.spid + ", запущенным с машины " +  
      b.hostname + ", выполняющим команду " + b.cmd  
FROM master..sysprocesses p  
      JOIN master..sysprocesses b ON b.spid = p.blocked
```

В данном запросе **master..sysprocesses** участвует дважды: один раз в качестве таблицы с информацией о блокируемом процессе (псевдоним **p**), второй раз - в качестве таблицы с данными о блокирующем процессе (псевдоним **b**).

Просто нужно понимать, что ЛОГИЧЕСКИ это две разные сущности и, соответственно, разные записи. То, что эти записи ФИЗИЧЕСКИ относятся к одной и той же таблице, ничего не меняет.

8. Внешнее объединение.

Вернемся к примеру со счетами и клиентами и рассмотрим следующий вариант заполнения таблиц **Clients** и **Accounts** данными:

Clients

ClientID	FirstName	LastName	DateBorn
1	Иван	Иванов	1980-12-10
2	Петр	Петров	1976-05-08

Accounts

AccountID	ClientID	Currency	DateOpen
26007111111001	1	UAH	2011-10-12
26003111111002	1	USD	2011-11-01

Как видно, у Петрова нет ни одного открытого счета. Результатом выполнения в описанной ситуации уже рассмотренного нами запроса

```
SELECT a.AccountID, a.Currency, c.LastName, c.FirstName
FROM Clients c JOIN Accounts a ON a.ClientID = c.ClientID
```

будет следующий resultset:

AccountID	Currency	LastName	FirstName
26007111111001	UAH	Иванов	Иван
26003111111002	USD	Иванов	Иван

Информация о Петрове отсутствует, что, в общем-то, логично: в **Accounts** нет записей, удовлетворяющих условиям объединения.

Но что делать, если в подобной ситуации все-таки нужно отобразить данные о клиентах, даже если у них нет счетов? Этот вопрос решается при помощи **внешнего объединения**. При таком типе объединения в resultset могут быть добавлены записи, не удовлетворяющие условиям объединения.

Различают два вида внешних объединений: **левое** и **правое**.

В случае **левого** внешнего объединения в resultset включаются ВСЕ строки из первой (левой, внешней) таблицы и строки из второй (внутренней) таблицы, удовлетворяющие условиям объединения. Для строк из первой таблицы, не имеющих совпадений со строками из второй, генерируется набор NULL значений.

В случае **правого** внешнего объединения выбираются ВСЕ строки из второй (правой, внешней)

таблицы и соответствующие им строки из первой (внутренней), а для непарных строк генерируется NULL.

Sybase ASE поддерживает для описания внешних объединений как синтаксис **ANSI**, так и **Transact SQL**.

Для варианта **ANSI** является таблицей первой (левой) или второй (правой) определяется по порядку перечисления этих таблиц в предложении **FROM**.

Левое внешнее объединение в варианте **ANSI** выглядит так:

```
SELECT a.AccountID, a.Currency, c.LastName, c.FirstName
FROM Clients c LEFT OUTER JOIN Accounts a ON a.ClientID = c.ClientID
```

правое - так:

```
SELECT a.AccountID, a.Currency, c.LastName, c.FirstName
FROM Accounts a RIGHT OUTER JOIN Clients c ON a.ClientID = c.ClientID
```

В варианте Transact SQL используется оператор *= (левое внешнее объединение):

```
SELECT a.AccountID, a.Currency, c.LastName, c.FirstName
FROM Clients c, Accounts a
WHERE c.ClientID *= a.ClientID
```

или оператор =* (правое внешнее объединение):

```
SELECT a.AccountID, a.Currency, c.LastName, c.FirstName
FROM Clients c, Accounts a
WHERE a.ClientID =* c.ClientID
```

Для **Transact SQL** "левость" и "правость" таблиц определяется по расположению соответствующих полей-операндов относительно оператора *= или =*. Порядок перечисления таблиц в блоке FROM не важен.

Также для **Transact SQL** существует ограничение, что внутренняя таблица внешнего объединения не может участвовать в обычном объединении.

В результате выполнения любого из четырех приведенных выше запросов будет получен вот такой результат:

AccountID	Currency	LastName	FirstName
26007111111001	UAH	Иванов	Иван
26003111111002	USD	Иванов	Иван
NULL	NULL	Петров	Петр

Вообще, с точки зрения читабельности и легкости понимания запроса, описание внешних объединений при помощи ANSI SQL предпочтительнее. Особенно, если связывание таблиц

происходит по нескольким полям.

Даже Sybase рекомендует пользоваться ANSI-стандартом внешнего объединения, поскольку Transact SQL расширение внешнего объединения может выдавать различные результаты в зависимости от версии сервера и имеет больше ограничений по сравнению с ANSI.

Справка.

Процедура **sp_helpjoins** выводит столбцы двух объединяемых таблиц или представлений, которые являются вероятными кандидатами объединения. Сначала **sp_helpjoins** проверяет, определены ли внешние или общие ключи на двух объединяемых таблицах. В случае неудачи применяются более общие критерии – совпадение пользовательских типов данных, затем совпадение имени и системных типов данных.

9. Внешние объединения и NULL.

Внесем дополнения в наш пример со счетами и клиентами. Допустим, в таблице **Accounts** присутствует поле **Comment** (некий комментарий к счету), допускающее значение **NULL**.

Запрос, выводящий данные обо ВСЕХ клиентах, в также комментарии к их счетам, можно записать так:

```
SELECT c.LastName, c.FirstName, a.Comment  
FROM Clients c LEFT OUTER JOIN Accounts a ON a.ClientID = c.ClientID
```

В результате можем получить, к примеру, вот такой resultset:

LastName	FirstName	Comment
Иванов	Иван	NULL
Петров	Петр	NULL

Вопрос: что означает **NULL** в поле **Comment** данного resultset? Однозначного ответа дать невозможно. Либо у клиента нет счетов и соответствующая запись в **Accounts** не была найдена, либо счет, все-таки, есть, просто в записи о нем не заполнено поле **Comment**.

Если понимание данной ситуации важно с точки зрения логики задачи, нужно либо модифицировать запрос (например, добавить в resultset поле AccountID, являющееся первичным ключом таблицы Accounts и не допускающее значений NULL и ориентироваться по нему), либо менять логику.

В любом случае, о возможности возникновения подобных ситуаций при использовании внешних объединений нужно помнить.

10. Подзапросы.

Допустим, есть таблица **Salary**, содержащая информацию об окладах сотрудников. В этой таблице имеются поля **Name** (имя сотрудника) и **Amount** (сумма оклада). Для простоты будем считать, что значение поля **Name** уникально.

Задача: узнать, кто зарабатывает больше Васи.

Фактически, данная задача распадается на две:

- узнать, сколько зарабатывает Вася
- получить список тех, кто зарабатывает больше него

Один из вариантов решения данной задачи - использование **подзапросов**, т.е., вложение одного запроса внутрь другого. При этом внутренний запрос возвращает информацию, которую использует внешний запрос.

Для нашей задачи запрос с подзапросом будет выглядеть так:

```
SELECT Name
FROM Salary
WHERE Amount > (SELECT Amount FROM Salary WHERE Name = 'Вася')
```

Внутренний запрос возвратит сумму оклада Васи. Это число будет использована при выполнении внешнего запроса для отбора тех, кто зарабатывает больше.

Подзапросы могут использоваться для формирования полей **resultset** или участвовать в условиях, описанных в предложении **WHERE**.

Синтаксически подзапрос обязательно заключается в круглые скобки.

Упрощенный синтаксис использования подзапросов таков:

```
SELECT список_выбора_запроса,
      ( SELECT [DISTINCT] список_выбора_подзапроса
        FROM список_таблиц_подзапроса
        WHERE условия_подзапроса )
FROM список_таблиц_запроса
WHERE { выражение { [NOT] IN | оператор_сравнения [ANY | ALL] }
      | [NOT] EXISTS }
      ( SELECT список_выбора_подзапроса
        FROM список_таблиц_подзапроса
        WHERE условия_подзапроса )
```

Рассмотрим варианты типы подзапросов и варианты их использования подробнее.

С точки зрения взаимодействия с внешним запросом различают два основных вида подзапросов: **коррелированные** и **некоррелированные**.

Некоррелированные подзапросы выполняются независимо от основного запроса. Обычно некоррелированный подзапрос выполняется один раз перед основным, а потом для основного запроса используется результат его работы.

В примере с Васей мы имели дело именно с некоррелированным подзапросом.

Коррелированные подзапросы получают данные из внешнего запроса и зачастую могут служить альтернативой объединению таблиц.

Например, если вспомнить пример с клиентами и счетами и поставить перед собой задачу вывести информацию обо всех счетах в гривне, а также фамилию клиента, которому принадлежит каждый счет, ее можно решить так:

```
SELECT a.AccountID,  
       ( SELECT c.LastName  
         FROM Clients c  
         WHERE c.ClientID = a.ClientID )  
FROM Accounts a  
WHERE a.Currency = 'UAH'
```

В данном примере сначала начнет выполняться основной запрос, будет сформирована выборка данных из таблицы **Accounts**, после чего для каждой записи из этой выборки будет выполнен подзапрос, возвращающий фамилию клиента из таблицы **Clients**.

Важно!

При использовании коррелированных подзапросов обязательно нужно указывать псевдонимы таблиц внешнего запроса. Иначе обратиться из подзапроса к "внешним" данным не получится.

11. Оператор EXISTS.

Еще одна задача, решаемая при помощи подзапросов - это задача проверки наличия той или иной информации.

Например, требуется получить список клиентов, не имеющих ни одного счета. Такая задача решается при помощи оператора **EXISTS**. Этот оператор принимает в качестве аргумента подзапрос и возвращает **TRUE**, если подзапрос вернул хотя бы одну запись и **FALSE**, если подзапрос вернул пустой resultset.

Описанная задача получения списка клиентов, не открывших ни одного счета, может быть решена так:

```
SELECT *
FROM Clients c
WHERE NOT EXISTS( SELECT 1 FROM Accounts a WHERE a.ClientID = c.ClientID )
```

Для каждой записи из таблицы **Clients** будет выполнен коррелированный подзапрос к таблице **Accounts**. В resultset внешнего запроса будут включены только те записи из **Clients**, для которых оператор **EXISTS** вернет значение **FALSE** (а **NOT EXISTS**, соответственно, **TRUE**). А такое значение **EXISTS** вернет только для клиентов, не имеющих ни одного счета (по ним подзапрос вернет пустой resultset).

Список выбора подзапроса, передаваемого оператору **EXISTS**, может содержать любое количество полей. Для **EXISTS** важен только факт наличия или отсутствия записей в возвращаемом подзапросом resultset.

12. Особенности использования подзапросов.

При использовании подзапросов нужно учитывать следующие ограничения.

- Список выбора подзапроса (за исключением подзапросов EXISTS) может содержать только **одно** имя столбца или одно выражение
- При сравнении данных поля основного запроса с результатом работы подзапроса их типы должны быть **совместимы**
- Для исключения неоднозначности необходимо указывать принадлежность полей к таблице
- Нельзя использовать подзапросы в предложении **ORDER BY**, **GROUP BY**, или **COMPUTE** (выражения GROUP BY и COMPUTE будут рассмотрены позже)
- Подзапросы не могут манипулировать своими результатами, т.е. подзапрос не может включать предложение **ORDER BY**, **COMPUTE** или **INTO** (выражения COMPUTE и INTO будут рассмотрены позже)
- Максимальный уровень вложенности подзапросов – **16**
- Подзапросы, связываемые с внешним запросом операторами сравнения, требующими единственного операнда, должны возвращать единственное значение
- В подзапросах нельзя использовать ключевое слово **TOP**

Также существуют рекомендации по использованию подзапросов:

- В Transact SQL подзапрос может использоваться везде, где может использоваться выражение
- Для исключения неоднозначности необходимо указывать принадлежность полей таблицам при помощи псевдонимов таблиц
- Лучше располагать подзапросы с правой стороны оператора сравнения (а иногда они должны там находиться обязательно)

13. Типы результатов подзапросов и их использование.

Как коррелированные, так и некоррелированные подзапросы в зависимости от типа формируемого результата можно разбить на следующие категории:

Single-row	Подзапросы, гарантированно возвращающие единственное значение в одной строке.
Multi-row	Подзапросы, способные вернуть несколько строк или пустой resultset
Exists	Подзапросы, представляющие из себя тест на существование данных

Single-row подзапросы могут быть использованы в качестве полей resultset, в скалярных выражениях или использоваться в блоке WHERE совместно с обычными операторами сравнения (=, >, <, != и т.д.):

```
SELECT Name
FROM Salary
WHERE Amount > (SELECT Amount FROM Salary WHERE Name = 'Вася')
```

Multi-row подзапросы используются совместно с оператором **IN** или модифицированными операторами сравнения.

Оператор **IN** позволяет проверить вхождение значения в список, формируемый подзапросом.

Модифицированные операторы сравнения - это операторы сравнения (=, >, <, != и т.д.), используемые совместно с ключевым словами **ALL** или **ANY**.

ANY говорит о том, что условие должно выполниться хотя бы для одного из значений в списке, **ALL** подразумевает, что условие должно выполниться для всех элементов списка.

Примеры.

```
-- Получение списка платежей, осуществленных в тех валютах,
-- в которых открыл счета клиент с ID 28:
```

```
SELECT *
FROM Payments p
WHERE p.Currency IN ( SELECT a.Currency
                     FROM Accounts a
                     WHERE a.ClientID = 28 )
```

```
-- или
```

```
SELECT *
FROM Payments p
WHERE p.Currency = ANY ( SELECT a.Currency
                       FROM Accounts a
                       WHERE a.ClientID = 28 )
```

```
-- Получение списка сотрудников, зарабатывающих больше, чем Вася и Петя:
```

```
SELECT s1.Name
FROM Salary s1
WHERE s1.Amount > ALL ( SELECT s2.Amount
                        FROM Salary s2
                        WHERE s2.Name IN ('Вася', 'Петя') )
```

Замечание.

Едва ли имеет смысл использовать модифицированные операторы сравнения **= ALL** или **!= ANY**. Для случаев возврата подзапросом resultset из более чем одной строки и наличия в нем разных значений первый оператор всегда вернет **FALSE**, а второй - **TRUE**.

14. Проблемы, возникающие при использовании подзапросов.

Наиболее частые проблемы, возникающие при использовании подзапросов - это возврат подзапросом `resultset` из более чем одной строки или пустого `resultset` в тех местах, где требуется `Single-row` запрос.

Следует помнить, что при использовании подзапросов в выражениях, в качестве полей `resultset` внешнего запроса или совместно с немодифицированными операторами сравнения нужно обеспечивать гарантированный возврат единственного значения.

15. Объединения или подзапросы?

Как уже демонстрировалось, подзапросы зачастую могут служить альтернативой объединению таблиц. Приведенные ниже запросы эквивалентны:

```
SELECT a.AccountID,  
      ( SELECT c.LastName  
        FROM Clients c  
        WHERE c.ClientID = a.ClientID )  
FROM Accounts a  
WHERE a.Currency = 'UAH'
```

```
SELECT a.AccountID, c.LastName  
FROM Accounts a JOIN Clients c ON c.ClientID = a.ClientID  
WHERE a.Currency = 'UAH'
```

Четких рекомендаций о том, какой из подходов лучше нет. Зачастую выбор того или иного пути определяется личными предпочтениями разработчика запроса. Но о существовании данной альтернативы следует помнить, иногда переход от одной формы к другой помогает при оптимизации сложных запросов.

16. Выборка из запроса.

Последние версии Sybase ASE позволяют использовать подзапрос в качестве источника информации в предложении **FROM**.

Например, это удобно использовать, если требуется связать таблицу с набором данных, формируемых сложным запросом.

В качестве примера рассмотрим ситуацию, когда нужно получить информацию о счетах и открывших эти счета клиентах, причем клиенты отбираются по принципу "мужчины, родившиеся после 1.01.1970 или женщины, родившиеся до 1.01.1990". Будем считать, что пол клиента содержится в поле **Sex** таблицы **Clients** (допустимые значения - '**M**' и '**F**', male - мужчина и female - женщина), а дата рождения клиента содержится в поле **DateBorn**.

Один из вариантов решения этой задачи:

```
SELECT a.AccountID, a.Currency, c.LastName, c.FirstName
FROM Accounts a,
    ( SELECT ClientID, LastName, FirstName
      FROM Clients
     WHERE Sex = 'M' AND DateBorn > '1970-01-01'
     UNION ALL
      SELECT ClientID, LastName, FirstName
      FROM Clients
     WHERE Sex = 'F' AND DateBorn < '1990-01-01' ) as c
WHERE a.ClientID = c.ClientID
```